SHARING · DOMAIN 4 - DATA SHARING AND FEDERATION

Delta Sharing

Section 4. Two protocols, one read-only promise, and what WITH HISTORY really exposes.

SECTION 4 of 10 WEIGHT 5% TOPIC Sharing

admin@sthithapragna.com

Delta Sharing is now OpenSharing

- Announced **10 June 2026**. Databricks' own release notes say plainly: Delta Sharing is now OpenSharing.
- The exam guide is dated **3 July 2026** and still says delta sharing throughout. Expect the old name.
- **Nothing breaks.** Existing shares keep working. It adds capability rather than replacing anything.
- What it adds: AI models, agent skills, unstructured data, and **Iceberg client support**.

> ON THE EXAM
>
> The protocol is now a Linux Foundation project. For the exam, the concepts are unchanged and the old name is the one you will see.

Nothing is copied. The recipient reads your files, with credentials you control and can revoke.

Three things you create

OBJECT	WHAT IT IS
Provider	You, when sharing. The metastore that holds the data.
Share	A named collection of objects from your metastore, made available together.
Recipient	A named object representing the person or organisation you are sharing with.

ON THE EXAM

Creating a recipient requires metastore admin, or the CREATE RECIPIENT privilege on the metastore. It is not an ordinary user action.

Databricks-to-Databricks against open sharing

D2D

Both sides on Databricks

- Recipient has a Unity Catalog workspace
- No token, no credential file, no activation link
- Shares views, volumes, models and notebooks too
- Identities are handled between metastores

D2O, OPEN SHARING

The recipient is anywhere

- Bearer token, or OIDC federation
- Credential file plus an activation link
- Delta-format tables
- Any tool that speaks the protocol

THE TIEBREAKER

The question that decides it is simply whether the recipient has a Unity Catalog-enabled Databricks workspace. If yes, use D2D; there is no reason not to.

How a non-Databricks recipient authenticates

- Either a **long-lived bearer token**, or **OIDC federation** through the recipient's own identity provider.
- With a token, Databricks generates a **credential file** and an **activation link**.
- The activation link must be sent over a **secure channel**. It is the only thing standing between the data and anyone.
- Token lifetime is configurable at recipient creation, and OIDC avoids a long-lived secret entirely.

> ON THE EXAM
>
> OIDC federation is the stronger option when available, because authentication stays with the recipient's identity provider rather than a token you issued.

Not everything can be shared both ways

ASSET	WHERE IT WORKS
Delta tables	Both protocols.
Views	Databricks-to-Databricks only.
Unity Catalog volumes	Databricks-to-Databricks only.
Unity Catalog models	Databricks-to-Databricks only.
Notebook files	Databricks-to-Databricks only.

ON THE EXAM

A requirement to share a view or a volume with an external partner is a requirement that open sharing does not meet.

WITH HISTORY is not only about history

- Sharing with history enables **cloud tokens**: temporary, path-scoped credentials to the table's storage.
- The recipient then reads the files **directly**, with performance the docs call comparable to direct access.
- **Default from DBR 16.2 and above.** Sharing a whole schema shares all its tables with history.
- Existing shares are not upgraded automatically. You must alter the share.

> ON THE EXAM
>
> The name undersells it. This is the setting that decides whether a recipient reads through the sharing server or straight from your storage.

What a cloud token actually grants

- Credentials are scoped to the table's **root directory**, giving read access to the data files **and the Delta log**.
- The log holds the **commit history**, **who committed each change**, and **deleted data not yet vacuumed**.
- So sharing with history can expose more than the table's current contents.
- If that matters, vacuum before sharing, or share without history and accept the performance cost.

> ON THE EXAM
>
> This is stated directly in the documentation and it is easy to miss. A performance setting has a data privacy consequence.

Partition filters, and what they cost

- A **partition specification** shares only part of a table, set when you add it to a share.
- That is how one table serves several recipients who should each see only their own slice.
- But on Databricks-to-Databricks, a partition filter makes the table **ineligible for cloud tokens**.
- So filtering and direct-read performance are in tension. You get one or the other.

> ON THE EXAM
>
> The tension makes sense: a credential scoped to a directory cannot enforce a row filter. Restricting what is visible means reading through the server.

Read-only, live, and on any platform

- **Shared data is read only.** A recipient cannot write back through a share.
- It is live. There is no refresh to schedule, because the recipient reads your current table.
- Recipients use their own tools: pandas, open-source Spark, Power BI, Tableau, or a Databricks workspace.
- That is the second objective's phrase: sharing live data to any computing platform.

> ON THE EXAM
>
> Live and read-only together are why sharing beats an export. No copy exists to drift, and no copy exists to leak.

Where sharing can and cannot reach

- Databricks-to-Databricks supports sharing **across regulatory domains**.
- Commercial workspaces can share with government clouds and with Azure China.
- Sharing between **Azure China and the government clouds is not supported**.
- These matrices change. Check the current support matrix rather than trusting a course.

> ON THE EXAM
>
> Cross-cloud and cross-region sharing works, which is often the actual reason an organisation chooses Delta Sharing over an export pipeline.

What to carry into the exam

- **D2D when the recipient has Unity Catalog**, open sharing otherwise. That single fact decides it.
- Views, volumes, models and notebooks are **D2D only**. Open sharing carries Delta tables.
- **WITH HISTORY enables cloud tokens** and direct reads. It also exposes the Delta log.
- A partition filter and cloud tokens are **mutually exclusive** on D2D.
- Shared data is **read only** and **live**. Nothing is copied.

> ON THE EXAM
>
> If an answer involves exporting, copying or scheduling a sync between two organisations, sharing is almost certainly the intended alternative.

A retailer must give a logistics partner access to a curated shipments table. The partner runs their own Databricks workspace with Unity Catalog enabled.

An engineer proposes creating an open sharing recipient, generating a bearer token, and emailing the activation link to the partner's data team.

What is the better approach?

A The proposal is correct; open sharing is the standard route for any external organisation.

B Use Databricks-to-Databricks sharing, which needs no token, credential file or activation link.

C Export the table nightly to a shared storage bucket the partner can read.

D Use open sharing but with OIDC federation rather than a bearer token.

◆ PAUSE AND DECIDE

What is the better approach?

A The proposal is correct; open sharing is the standard route for any external organisation.

External does not mean non-Databricks. The partner has Unity Catalog, so D2D applies.

B Use Databricks-to-Databricks sharing, which needs no token, credential file or activation link.

The recipient has a Unity Catalog workspace, which is the only fact that decides the protocol.

C Export the table nightly to a shared storage bucket the partner can read.

Creates a copy that goes stale and cannot be revoked. Sharing exists to avoid this.

D Use open sharing but with OIDC federation rather than a bearer token.

Better than a bearer token, and still the wrong protocol for this recipient.

ON THE EXAM

The distinction is not internal against external. It is whether the recipient has Unity Catalog.

> A healthcare provider shares a patient events table with an analytics partner using WITH HISTORY for performance. Records are routinely deleted from the source table when patients exercise their right to erasure.
>
> A privacy review asks whether deleted records could still be visible to the recipient.

What is the correct answer?

A. No. Deletes are applied immediately and the recipient sees only the current table.

B. Yes. Cloud tokens grant access to the Delta log, which retains deleted data until it is vacuumed.

C. No. The Delta log contains metadata only, never data.

D. Yes, but only if the recipient also has the CREATE RECIPIENT privilege.

◆ PAUSE AND DECIDE

What is the correct answer?

A. No. Deletes are applied immediately and the recipient sees only the current table.

True of a query against the table. Not true of direct file access to its directory.

B. Yes. Cloud tokens grant access to the Delta log, which retains deleted data until it is vacuumed.

Stated in the docs: the log contains commit history, committer information, and deleted data not yet vacuumed.

C. No. The Delta log contains metadata only, never data.

The log references data files that still exist until vacuum removes them.

D. Yes, but only if the recipient also has the CREATE RECIPIENT privilege.

That privilege is about creating recipients on the provider side. It is unrelated.

ON THE EXAM

A performance setting with a data privacy consequence. In a regulated context, vacuum before sharing, or share without history.

A data team maintains a curated view that joins three tables and masks two columns. They need to make this view available to an external partner who uses Snowflake, not Databricks.

They add the view to a share and create an open sharing recipient. The view does not appear for the recipient.

What is happening, and what should they do?

A Views are Databricks-to-Databricks only. Materialise the view as a table and share that.

B The view needs to be shared WITH HISTORY before it becomes visible.

C The recipient must install a Delta Sharing connector before views appear.

D Views require the recipient to be granted SELECT on the underlying tables as well.

◆ PAUSE AND DECIDE

What is happening, and what should they do?

A Views are Databricks-to-Databricks only. Materialise the view as a table and share that.

Open sharing carries Delta tables. A view needs a Unity Catalog on the other side to resolve against.

B The view needs to be shared WITH HISTORY before it becomes visible.

WITH HISTORY concerns cloud token eligibility for tables, and views are not eligible at all.

C The recipient must install a Delta Sharing connector before views appear.

A connector is needed to read anything, and it does not make an unsupported asset type appear.

D Views require the recipient to be granted SELECT on the underlying tables as well.

That would defeat the masking the view exists to provide.

ON THE EXAM

Materialising the view is not a workaround, it is the intended pattern. The masking is applied when the table is written.

> A provider shares one large transactions table with four regional partners. Each partner must see only their own region, so the table is added to each share with a partition specification.
>
> Partners report slower query performance than a similar unfiltered share they receive from another provider.

What explains the difference?

A Partition filters are evaluated by the recipient, so they always cost more.

B A partition filter makes the table ineligible for cloud tokens, so reads go through the sharing server rather than directly to storage.

C The table needs OPTIMIZE run before each share refresh.

D Four shares of the same table cause contention on the provider's storage.

◆ PAUSE AND DECIDE

What explains the difference?

A Partition filters are evaluated by the recipient, so they always cost more.

Backwards. The filter exists so the recipient never sees the other partitions.

B A partition filter makes the table ineligible for cloud tokens, so reads go through the sharing server rather than directly to storage.

A credential scoped to a directory cannot enforce a row filter, so filtered shares lose direct file access.

C The table needs OPTIMIZE run before each share refresh.

File layout is worth attention generally, and it does not explain the difference against an unfiltered share.

D Four shares of the same table cause contention on the provider's storage.

Invented. Shares do not contend with one another.

ON THE EXAM

This is a genuine trade-off rather than a misconfiguration. Restricting visibility and granting direct file access are incompatible by design.

> A team creates an open sharing recipient with a bearer token and emails the activation link to a distribution list at the partner organisation. Some weeks later, a security review finds the link was forwarded outside the intended team.
>
> The partner's organisation runs its own identity provider.

What is the strongest remediation?

A Regenerate the token and email the new activation link to a narrower list.

B Move the recipient to OIDC federation, so authentication is handled by the partner's identity provider.

C Shorten the token lifetime and keep the current process.

D Switch to Databricks-to-Databricks sharing to remove the token.

◆ PAUSE AND DECIDE

What is the strongest remediation?

A. Regenerate the token and email the new activation link to a narrower list.

Repeats the same process and hopes for a better outcome. The mechanism is unchanged.

B. Move the recipient to OIDC federation, so authentication is handled by the partner's identity provider.

Removes the long-lived secret entirely. Access follows the partner's own identity management, including leavers.

C. Shorten the token lifetime and keep the current process.

Reduces the window and keeps a forwardable secret at the centre of it.

D. Switch to Databricks-to-Databricks sharing to remove the token.

Only available if the partner has a Unity Catalog workspace, which nothing here states.

ON THE EXAM

The problem is not the token's length of life. It is that a bearer token is a secret that can be forwarded by anyone who holds it.

A share was created two years ago and carries a large fact table. A recipient on Databricks reports that queries against it are markedly slower than against a table shared by a newer provider.

Both are Databricks-to-Databricks shares of unfiltered managed Delta tables on current runtimes.

What is the most likely cause?

A The older share predates WITH HISTORY being the default, and was never altered to enable it.

B The older share needs to be recreated, since shares expire after two years.

C The recipient is reading a stale copy that needs refreshing.

D The older table needs its statistics recomputed on the provider side.

◆ PAUSE AND DECIDE

What is the most likely cause?

A The older share predates WITH HISTORY being the default, and was never altered to enable it.

WITH HISTORY became the default from DBR 16.2, and existing shares are not upgraded automatically.

B The older share needs to be recreated, since shares expire after two years.

Shares do not expire. Recipient tokens can, which is a different thing.

C The recipient is reading a stale copy that needs refreshing.

Nothing is copied. The recipient always reads the current table.

D The older table needs its statistics recomputed on the provider side.

Would affect both shares equally, and does not explain a difference between them.

ON THE EXAM

Defaults change, and existing objects keep the behaviour they were created with. That gap is a recurring source of unexplained differences.